vertex `v` at distance `d`; therefore, `v` is extracted from the queue after all vertices at distance `d-1` and before all vertices at distance `d+1`; from this, it follows that none of the vertices in the queue can have distance `d+2`, because no vertex at distance `d+1` has been visited yet, and we only add vertices to the queue when examining each visited vertex' adjacency list (so, the distance of a vertex added can only grow by `1` unit with respect to its parent). In turn, this guarantees that `u` will be visited before any vertex at distance `d+2` (or further away) from the source.

For the inductive hypothesis, moreover, we are sure that all vertices at distance `d-1` are visited before `v`, so all vertices at distance `d` are already in the queue, and hence they will be visited before `u`.

This property allows us to use a simple queue instead of a priority queue: not bad, considering that the former has a `O(1)` worst-case running time for enqueuing and dequeuing its elements (while heaps, for example, need `O(log(n))` steps for each operation), allowing us to keep the running time of BFS linear: it's a worst-case `O(|V| + |E|)`, linear in the largest between the number of vertices and edges. For connected graphs, this can be simplified to `O(|E|)`.

We mentioned that checking if search has reached the goal (line #10) and the whole concept of goal vertex are optional: we can also use BFS to just compute the paths and distances from a single source to all the other vertices in the graph[5]. It's worth noting that, as of today, there is no known algorithm that can find the shortest path to a single destination more efficiently than BFS: in other words, it's